

DSC 40BR: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

INTRODUCTION



HI, I'M BRIAN

HOW YOU'RE EVALUATED

Homework – 65%

- One homework per week
- Lowest will be dropped
- Released on Tuesdays
- Due on Fridays

Final exam – 35%

- Date: [TBD]
- Technical interview style
- Will upload a demo
- Practice exam will be given

WHY ALGORITHMS MATTER?

Your code runs in 5 seconds on 1,000 data points.

What happens if you now have a dataset with 100,000 points.

How long will it take?

500 S

100X THE DATA.
SIMPLE.

≈ 14 HOURS

SOME ALGORITHMS
SCALE MUCH WORSE

LONGER THAN
YOU HAVE

(SOME SCALE *THAT
BADLY*)

All three are real answers for real algorithms. Which one you get is a property of the algorithm, not the machine.

THE STOPWATCH CAN'T ANSWER THIS

- A timing is a fact about **your machine**, not about the algorithm.
- A timing is a fact about **one input**. Some inputs are easy, some are hard.
- A single timing says nothing about **scaling**. Scaling is the whole question.

We want to answer the question **before** we run the code, on **any** machine, for **every** input size.

THREE QUESTIONS

- How do we assess an algorithm's efficiency *without running it*?
- Why and how do the common algorithms work?
- How do we design faster ones with better strategies and better data structures?

Two pillars

PILLAR 1 – CORRECTNESS

Does it produce the right answer for every input, not just the ones we tested?

Tools: loop invariants, induction, proof by contradiction.

PILLAR 2 – EFFICIENCY

Is it fast enough as the data grows – and can we say so in advance?

Tools: time complexity analysis, recurrences

A correct algorithm that is too slow is useless. A fast algorithm that is wrong is worse.

WHERE IS DSC 40B GOING?

WEEK TOPIC

VALUE

- | | |
|---|---------------------------------|
| 1. Complexity, best/avg/worst case | The measuring instrument |
| 2. Search, sorting, Quickselect, BSTs | Divide & conquer, recurrences |
| 3. Hashing, graph theory, BFS | Structures that change the cost |
| 4. Shortest paths: BFS, DFS, Bellman-Ford, Dijkstra | The workhorse algorithms |
| 5. Prim, Kruskal, and what ' <i>hard</i> ' means* | Greedy design, and its limits |

*Time permitting

HOW TO WORK IN THIS COURSE

- Read the notes alongside the videos

The book is the reference; these modules are the TL;DR

- Write proofs by hand before checking

Reading a proof and producing one are different skills. Exams test the second.

- Start homework early enough to be stuck

Being stuck is the work. Slip days don't create understanding.

DECONSTRUCTING A PROBLEM

EVERY PROBLEM HAS FOUR LAYERS

1. The objective

A goal, in English. Usually vague.

2. The computational problem

A precise **GIVEN/COMPUTE** statement.

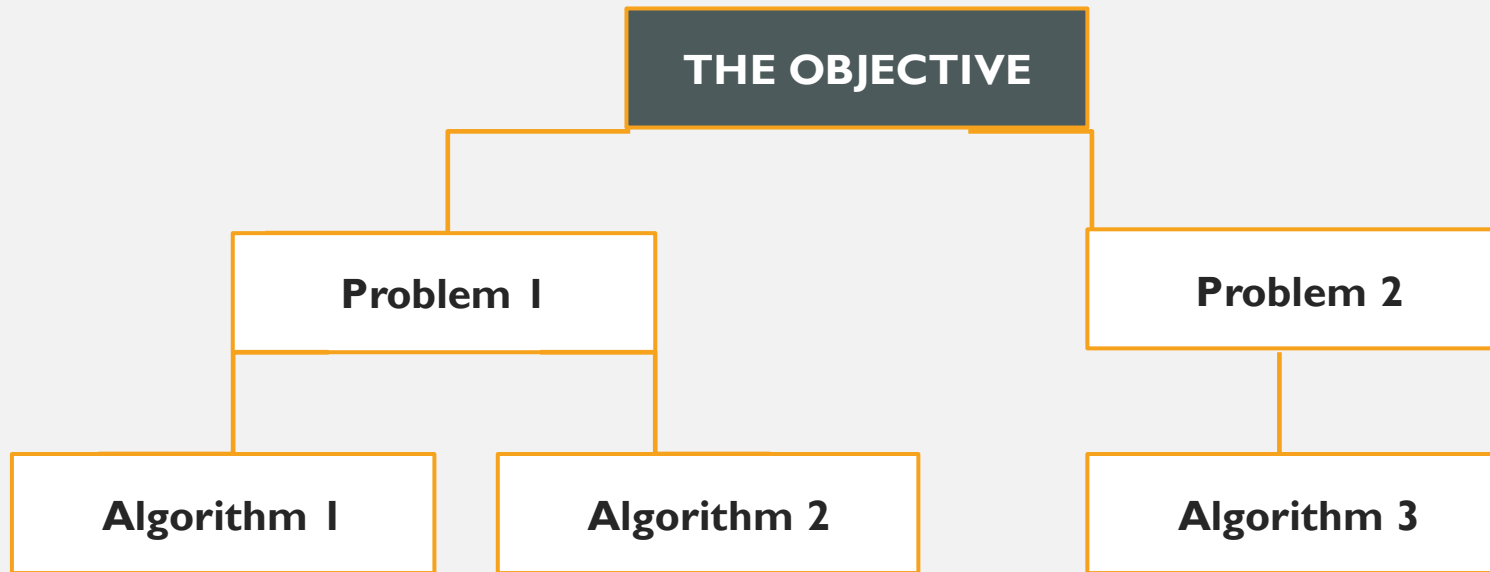
3. The algorithm

A procedure that solves it.

4. The implementation

Code that runs at the scale you have.

A TREE, NOT A PIPELINE



Choosing a path down the tree solves the problem. **Walking back up** it is how you interpret what you got.

THREE SPECIALISTS, ONE GENERALIST

STATISTICIAN

Poses the computational problem, and studies what its solutions mean.

COMPUTER SCIENTIST

Designs and analyses the algorithm that solves it.

ENGINEER

Makes the implementation run at scale, reliably.

A data scientist has to move between all three.

When the algorithm is too slow, the fix is often to go back up and pose a different problem.

THE OBJECTIVE

“Summarize this pile of numbers with one number”

That statement is not yet something a computer can do. There is no definition of summarize, and no way to check whether an answer is right.

THE COMPUTATIONAL PROBLEM

GIVEN: Real numbers $x_1 \dots x_n$

COMPUTE: Their mean, $\frac{1}{n} \sum x_i$

Now an answer can be right or wrong.

Notice we chose the mean. Choosing the median is a different branch of the tree.

THE ALGORITHM

- 1. Start a running total at zero.**
- 2. Traverse the list once, adding each number to the total.**
- 3. Divide the total by how many numbers there were.**

Language-independent. This is the thing we prove correct, and the thing whose cost we measure.

THE IMPLEMENTATION

```
def mean(data):  
    """Assume len(data) > 0."""  
    total = 0  
    for x in data:  
        total += x  
    return total / len(data)
```

New questions appear here:

What if the list is empty?

What about roundoff when one number is huge and another tiny?

Those are layer-4 problems. One of them comes back in the next module.

IDENTIFY WHICH LAYER YOU'RE STUCK ON

"I don't know what I'm optimizing."

Layer 1 $\rightarrow$ 2. Write a GIVEN / COMPUTE.

"I know what I want, not how to get it."

Layer 2 $\rightarrow$ 3. You need an algorithm.

"It's wrong sometimes."

Layer 3, correctness. Find the invariant.

"It works but it's too slow."

Layer 4, efficiency — or pose a different problem.

- | | |
|------------------------------|---------------------------------------|
| 1. The objective | A goal, in English. Usually vague. |
| 2. The computational problem | A precise GIVEN/COMPUTE statement. |
| 3. The algorithm | A procedure that solves it. |
| 4. The implementation | Code that runs at the scale you have. |

WHERE THIS GOES NEXT

We have an algorithm.

How do we know it's correct?

Testing shows the presence of bugs, never their absence. For loops, the tool is the loop invariant.

CORRECTNESS:
LOOP INVARIANTS

A CAUTIONARY TALE

TIMSORT WAS BROKEN FOR 13 YEARS

It is the sort behind Python's `sorted()` and Java's `Collections.sort()`.

In 2015 a team of computer scientists proved it contained a logical error.

In Python it only triggered on inputs nobody had ever passed in.

In Java on Android it failed on lists as small as 65,536 elements.

Thirteen years of passing test suites.

WHAT WE ACTUALLY WANT

Tests show that bugs are **present**.

They never show that bugs are **absent**.

WHAT A TEST PROVES: “SYSTEM
BUGS”

Correct for the inputs I tried

WHAT WE WANT TO PROVE:
“ALGORITHM BUGS”

Correct for every possible
input.

WHAT A LOOP INVARIANT IS

DEFINITION

A statement that is true before the loop runs, and true again immediately after every iteration.

A promise the loop keeps to itself.

If it holds at the start and survives one arbitrary iteration, it survives all of them, and we cash it in when the loop exits.

A USEFUL INVARIANT PAYS OUT AT THE END

TRUE BUT USELESS

"total is a number."

Invariant, yes. Tells you nothing about the answer.

USEFUL

"After iteration α , total is the sum of the first α elements."

Set $\alpha = n$ and you have the correctness claim.

Design it backwards, start from what must be true when the loop exits.

THE "ZEROth ITERATION"

CONVENTION

"Immediately after the 0_{th} iteration" means **immediately before the first iteration.**

The moment after iteration $\alpha-1$ is the moment before iteration α .

THIS IS JUST INDUCTION

BASE CASE

Show it holds before the first iteration.

INDUCTIVE HYPOTHESIS

Assume it holds for iteration $\alpha-1$.

INDUCTIVE STEP

Prove it holds after iteration α .

AND THEN...TERMINATE

Show that the loop ends, then read off what the invariant says at the last iteration. That is what proves the algorithm works.

WHAT WE'RE PROVING

```
def mean(data):  
    total = 0  
    for x in data:  
        total += x  
    return total / len(data)
```

INVARIANT

After the α_{th} iteration, total contains the sum of the first α elements of data.

Everything in the return value depends on total- so proving the loop proves the algorithm.

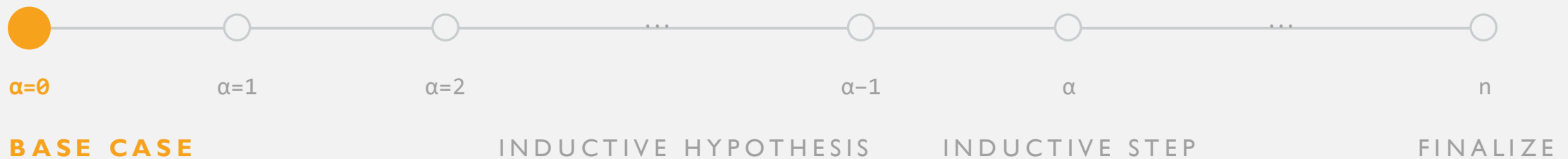
BASE CASE

Plug $\alpha = 0$ into the invariant.

total = 0 has just run, so total is 0.

By convention, the sum of zero elements is 0.

Base case holds.



INDUCTIVE HYPOTHESIS

Let α be an arbitrary iteration. Assume it holds after $\alpha-1$.

total enters holding the sum of the first $\alpha-1$ elements.

This iteration is just an assumption that $\alpha-1$ holds.

Inductive hypothesis holds.



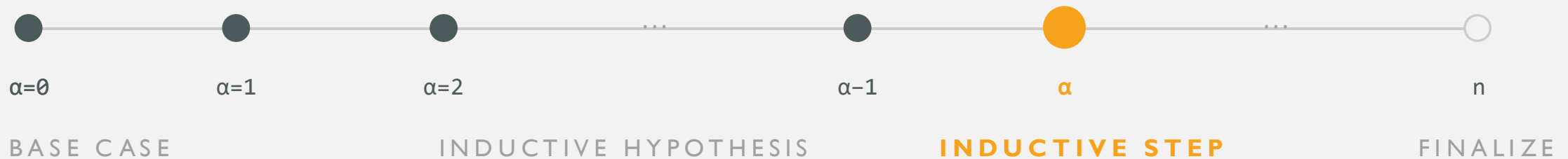
INDUCTIVE STEP

Let α be an arbitrary iteration. We know it held after $\alpha-1$.

total enters holding the sum of the first $\alpha-1$ elements.

This iteration adds x , the α_{th} element.

Inductive step holds.



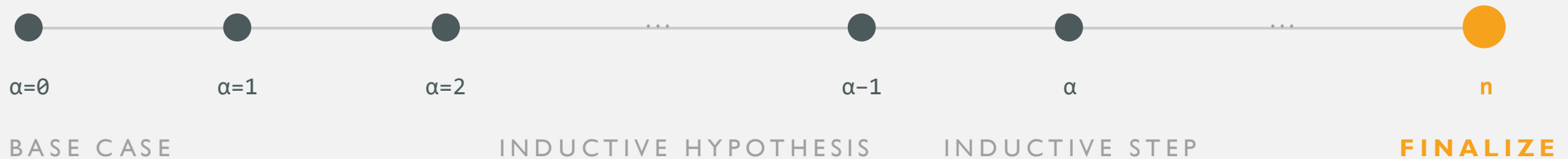
FINALIZE

Base case plus inductive step means it holds at every α .

Does the loop end? Yes. Once per element, so after n iterations.

At $\alpha = n$, total is the sum of all n elements.

We return $\text{total} / \text{len}(\text{data})$ - the algorithm is correct.



α IS NOT i

α : MATHEMATICAL VARIABLE

Counts iterations the way humans do. The first iteration is $\alpha = 1$. Lives only in the proof.

i : IMPLEMENTATION VARIABLE

Whatever the code says. In Python it usually starts at 0. Lives only in the program.

```
for i in range(10): print(i)
```

On the α th iteration this prints $\alpha - 1$.

TWO LOOPS NEED TWO INVARIANTS

```
def array_mean(arr):  
    total = 0  
    n = len(arr)  
    for i in range(n):  
        row_total = 0  
        for j in range(n):  
            row_total += arr[i, j]  
        total += row_total  
    return total / n**2
```

To prove the outer loop we need `row_total` to be the sum of row α .

But `row_total` is itself computed by a loop.
We can't just assert it.

PROVE INNER → USE IT TO PROVE OUTER

PROVE THE INSIDE FIRST

INNER INVARIANT

After the β_{th} iteration of the inner loop, during the α th iteration of the outer loop, `row_total` contains the sum of the first β elements of row α .

Base case `row_total` = 0 at the top of each outer iteration.

IH Assume `row_total` contains the sum of the first $\beta-1$ elements of row α .

Ind Step At $\beta = n$: `row_total` contains the sum of the first n elements of row α , i.e., `row_total` (from $\beta-1$) plus the n th element.

NOW THE OUTSIDE

OUTER INVARIANT

After the α th iteration of the outer loop, total contains the sum of the first α rows.

Base case total = 0 before the loop; the sum of zero rows is 0.

IH Assume total contains the sum of the first $\alpha-1$ rows. The inner loop's termination gives us the sum of row α .

Ind Step At $\alpha = n$, total contains the sum of every row. Divide by n^2 to get the return value.

The inner proof's payout becomes a single line in the outer proof.

YOUR TURN

```
def find_minimum(arr, start):  
    n = len(arr)  
    min_value = arr[start]  
    min_ix = start  
    for i in range(start + 1, n):  
        if arr[i] < min_value:  
            min_value = arr[i]  
            min_ix = i  
    return min_ix
```

Pause here. Write it by hand first.

State a loop invariant, then prove it.

WATCH FOR

The function returns an index, not a value. An invariant about `min_value` alone won't get you there.

SOLUTION

```
def find_minimum(arr, start):  
    n = len(arr)  
    min_value = arr[start]  
    min_ix = start  
    for i in range(start + 1, n):  
        if arr[i] < min_value:  
            min_value = arr[i]  
            min_ix = i  
    return min_ix
```

INVARIANT

After the α th iteration, `min_ix` contains the index of the minimum element in α elements of data.

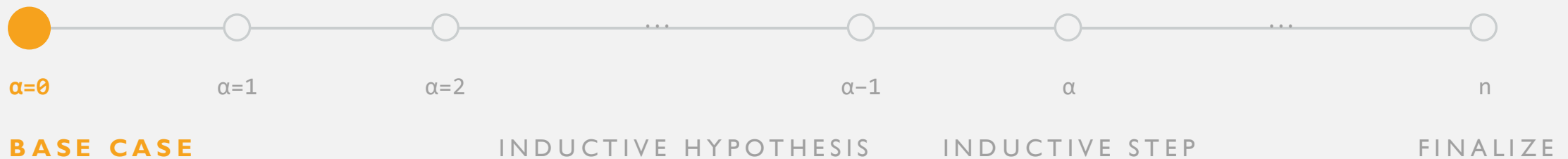
BASE CASE

Plug $\alpha = \text{start}$ into the invariant.

`min_ix = start` has just run.

Before the loop, `min_ix = start`. `arr[start..start]` is a length-1 range, so just one element.

Base case holds.



INDUCTIVE HYPOTHESIS

Let α be an arbitrary iteration. Assume it held after $\alpha-1$.

`min_ix` enters holding the index of the min of the first $\alpha-1$ elements.

`min_ix` = index of min of `arr[start...  $\alpha-1$ ]`

So `min_ix` is now the min index of the first α — inductive hypothesis holds.



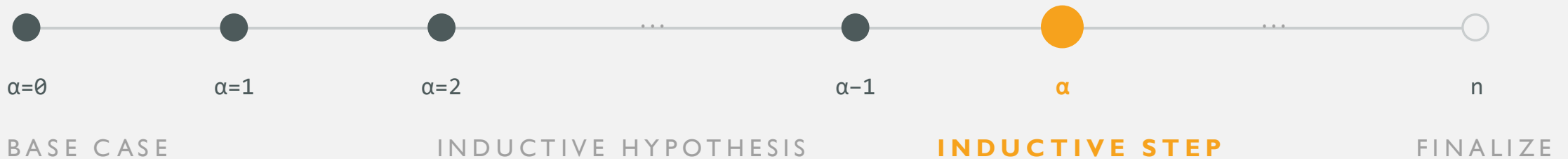
INDUCTIVE STEP

Let α be an arbitrary iteration. Assume it held after $\alpha-1$.

`min_ix` enters holding the index of the min of the first $\alpha-1$ elements.

If `arr[ $\alpha$ ] < min_value`: `min_ix =  $\alpha$` . This iteration compares `arr[ $\alpha$ ]` against min of `arr[start...  $\alpha-1$ ]`.

So `min_ix` is now the min of the first α — inductive hypothesis holds.



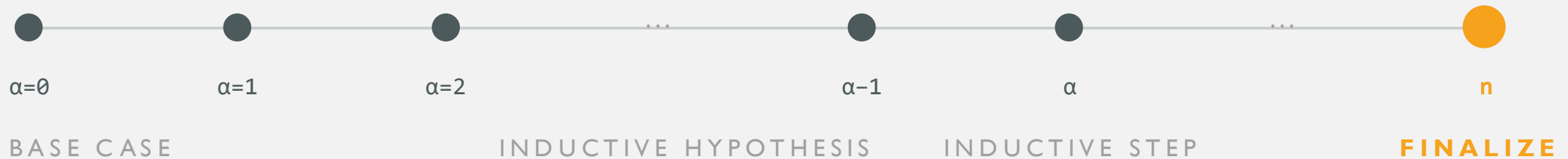
FINALIZE

Base case plus IH means it holds at every α .

Does the loop end?

At $\alpha = n$, `min_ix` is the index of the min of `arr[start...n-1]`.

We returned `min_ix`. The algorithm is correct.



WHAT YOU CAN DO NOW

Take a problem apart into its four layers.

State a loop invariant that pays out at termination.

Prove it by induction.

Handle a nested loop by handing the inner result to the outer.

NEXT — LECTURE 2, COMPLEXITY I. 5 seconds on 1,000 points. How long on 100,000?